

WandiSSAL_LoggerAppsMsg()

Synopsis

```
#include "WandiSSAL-LoggerLib-2.1.h"

int  WandiSSAL_LoggerAppsMsg(int devProd, int message_group, int severity,
char *message_string, ... )
```

Description

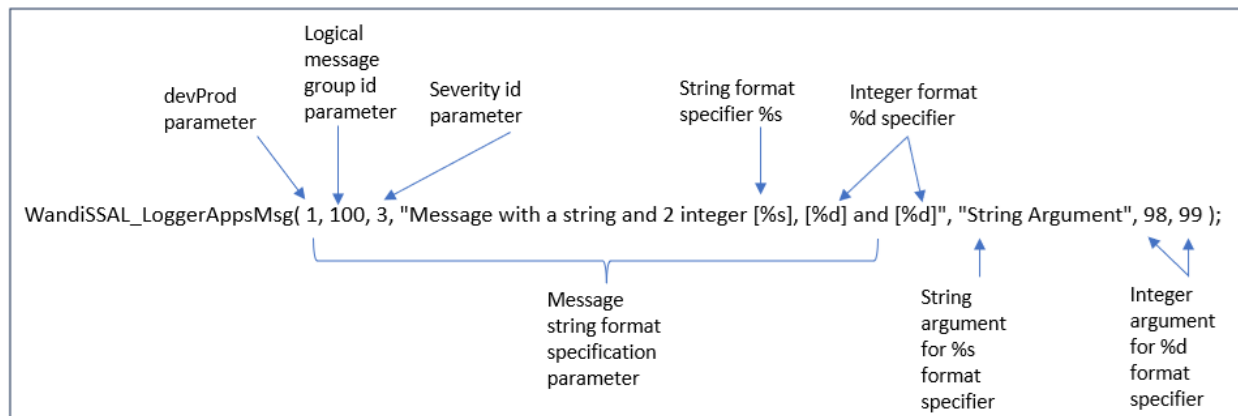
WandiSSAL_LoggerAppsMsg() decomposes argument parameters into id=value pairs so that it can validate for errors, encode id=value pairs to reduce storage capacity, and securely protects id=value pairs to ensure confidentiality, integrity, and authenticity.¹

Example Usage

```
WandiSSAL_LoggerAppsMsg(1,100,3,"Message that generates 1 string and 2
integers [%s], [%d] and [%d]","String Argument", 98,99);
```

Generates formatted output log message:

```
02/09/2026 12:01:55 [3650:sample_program] [DATABASE] [INFORM] Message that
generates 1 string and 2 integers [String Argument], [98] and [99]
```



The `devProd` integer parameter 1 indicates a development type log message and 2 indicates a production type log message. If the `PRODUCTION='n'` in the `LoggerMessageDefines.txt` file then log messages with `devProd=1` and `devProd=2` will be generated. If the `PRODUCTION='y'` in the `LoggerMessageDefines.txt` file then only log messages with `devProd=2` will be generated. The benefit of this is to prevent generating unnecessary development log messages in a production environment.

The logical `message_group` integer parameter is the id part of the id=value pairs defined in file `LoggerMessageDefines.txt`. The id part of 100 is translated and mapped to the corresponding value

¹ This free version only supports decompose and validate format string specification and string argument parameters.

part of DATABASE in the output message. Refer to the `LoggerMessageDefines.txt` document for further details.

The `severity` integer parameter is the id part of the id=value pairs defined in file `LoggerMessageDefines.txt`. The id part of 3 is translated and mapped to the corresponding value part of INFORM in the output message. Refer to the `LoggerMessageDefines.txt` document for further details.

The `message_string` format string specification parameter is a C-Programming Language `printf()` style format string specification that comprises of format string text and format specifiers. This `message_string` parameter is bounded by the guidelines in the `WandiSSAL-Bound-Capabilities.pdf` document.

The reminder of the `WandiSSAL_LoggerAppsMsg()` parameters (as indicated by "...") represents the corresponding format string specifiers argument values for `message_string`.

NOTE: Refer to the `WandiSSAL-Bound-Capabilities.pdf` document for validation guidelines used by `WandiSSAL_LoggerAppsMsg()` to check and validate parameter values.

`WandiSSAL_LoggerAppsMsg()` returns 0 on success and -1 on error.

This API function is failed safe which means when an error occurs the application can continue processing.

Error Codes

Error codes are written to the following files, where the `<cpid>` is the application process id number:

- `/tmp/<cpid>-LoggerAppsMessage.txt`
- `$LOGGER_HOME_PATH/LoggerOutput/<cpid>-LoggerAppsMessage.txt`

135	Invalid <code>devProd</code> parameter. Solution: Refer to <code>devProd</code> guidelines above.
136	Invalid <code>message_group</code> , <code>severity</code> and <code>message_string</code> parameter. Solution: Refer to <code>message_group</code> and <code>message_string</code> guidelines above.
137	Invalid <code>message_string</code> parameter. Solution: Refer to <code>message_string</code> guidelines above
138	Too many format string type specifiers. Solution: Refer to <code>WandiSSAL-Bound-Capabilities.pdf</code> guidelines.
139	String argument pointer parameter is NULL. Solution: Make sure string argument pointer is not NULL.
140	String argument pointer is -1. Solution: Make sure string argument pointer is not -1.
141	String argument not properly NULL terminated. Solution: Refer to <code>WandiSSAL-Bound-Capabilities.pdf</code> guidelines.
142	String argument exceeded allowed length of 64 characters. Solution: Refer to <code>WandiSSAL-Bound-Capabilities.pdf</code> guidelines.
143	Invalid string argument value. Solution: Refer to <code>WandiSSAL-Bound-Capabilities.pdf</code> guidelines.
144	Invalid format string specifier type. Solution: Refer to <code>WandiSSAL-Bound-Capabilities.pdf</code> guidelines.

115, 117	Parameter <code>message_string</code> is NULL or exceeded allowed length of 256 characters. Solution: Refer to <code>WandiSSAL-Bound-Capabilities.pdf</code> guidelines.
104, 106, 116	Invalid ASCII character in <code>message_string</code> or string argument. Solution: Refer to <code>WandiSSAL-Bound-Capabilities.pdf</code> guidelines.
105	String argument length is zero. Solution: Make sure string argument is not zero length.